

From-Scratch Implementation of the Transformer Architecture for English-Hindi Machine Translation

Kushagra Singhal, Roll No. 240150046
Course: DA221, April 2026

Abstract—This paper presents an end-to-end, from-scratch neural machine translation pipeline for English-to-Hindi based on the seminal “Attention Is All You Need” architecture. Dispensing entirely with traditional recurrence and convolutions, the implemented Transformer utilizes multi-head self-attention, SentencePiece Byte-Pair Encoding (BPE), label smoothing, and Noam scheduling. To accommodate strict local computational constraints—specifically an NVIDIA RTX 2050 GPU with 2 GB VRAM—the model dimensionality was scaled down and trained on a 30,000-pair synthetic dataset. We provide reproducible training and multi-strategy inference pipelines, including both greedy and beam search decoding. While the architecture is functionally sound and achieves a BLEU-4 score of 7.22, the limited scale highlights the steep hardware capacity, memory bandwidth, and data requirements necessary for convergence in modern large-scale language modeling.

Index Terms—Transformer, Machine Translation, Self-Attention, Natural Language Processing, English-to-Hindi, BPE, Beam Search.

I. INTRODUCTION

Historically, the dominant sequence transduction models in natural language processing (NLP) were based on complex recurrent neural networks (RNNs) or convolutional neural networks (CNNs) operating within an encoder-decoder framework. While architectures utilizing Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) drove significant advancements in machine translation, their inherently sequential nature presented a fundamental limitation. Recurrent models factor computation along the symbol positions of the input and output sequences, generating a sequence of hidden states h_t as a function of the previous hidden state h_{t-1} . This sequential dependency precludes parallelization within training examples, becoming a critical computational bottleneck at longer sequence lengths due to memory constraints.

To overcome this, Vaswani et al. [1] proposed a simple yet revolutionary network architecture: the Transformer. By eschewing recurrence and relying solely on attention mechanisms, the Transformer established a new paradigm capable of drawing global dependencies between input and output sequences in parallel.

The objective of this project is to implement the complete Transformer architecture from scratch, using PyTorch primitives, to thoroughly understand its underlying mathematical mechanics and data routing. We apply this architecture to an English-to-Hindi (EN-HI) translation task. This report details

the synthesis of the foundational theory with practical implementation engineering, managing the entire pipeline from dataset generation and subword tokenization to interactive beam-search inference, while critically evaluating the model’s performance under constrained local computational resources.

II. RELATED WORK

Prior to the advent of the Transformer, sequence modeling and transduction problems were dominated by recurrent models. Efforts to push the boundaries of encoder-decoder architectures frequently incorporated attention mechanisms to allow the modeling of dependencies without regard to their distance in the sequences, most notably pioneered by Bahdanau et al. [2]. However, these additive attention mechanisms were almost exclusively used in conjunction with a recurrent network.

The goal of reducing sequential computation also drove the development of architectures like the Extended Neural GPU and ConvS2S, which utilized CNNs as basic building blocks to compute hidden representations in parallel. However, in these models, the number of operations required to relate signals from two arbitrary input or output positions grows linearly or logarithmically with the distance between positions, making it difficult to learn long-range dependencies.

The Transformer addressed this by reducing the number of operations to a constant, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions—an effect counteracted by Multi-Head Attention.

III. MATHEMATICAL FORMULATION AND PROBLEM SETUP

Given a source sentence $x = (x_1, x_2, \dots, x_n)$ in English, the fundamental objective is to model the conditional probability of a target sentence $y = (y_1, y_2, \dots, y_m)$ in Hindi. The model calculates this auto-regressively as:

$$P(y|x) = \prod_{t=1}^m P(y_t | y_{<t}, x) \quad (1)$$

The model is trained to maximize this likelihood by minimizing token-level cross-entropy.

IV. METHODOLOGY: THEORY AND IMPLEMENTATION

A. Data Preparation and Tokenization

In production-grade NLP, Transformers require massive parallel corpora. To train the model within local hardware limits, a deterministic dataset of 33,000 parallel EN-HI sentence pairs

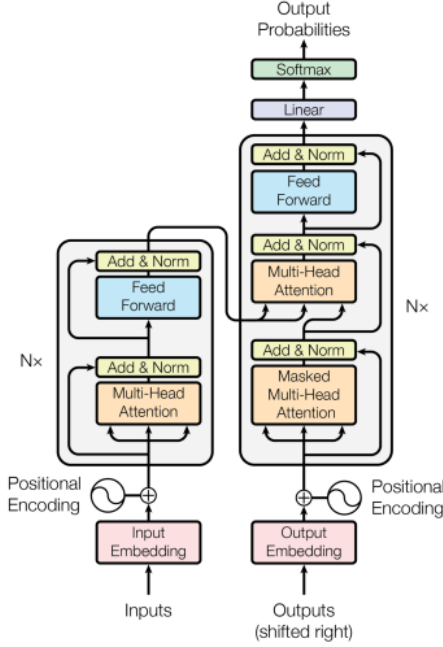


Fig. 1: The Transformer model architecture, consisting of stacked Encoder and Decoder layers with multi-head attention and feed-forward networks [1].

was generated synthetically using compositional templates and phrase-based generation. The corpus was split into 30,000 training pairs, 1,500 validation pairs, and 1,500 test pairs.

Instead of basic word-level tokenization, we implemented SentencePiece Byte-Pair Encoding (BPE), building upon the subword unit modeling demonstrated by Sennrich et al. [3]. BPE effectively mitigates out-of-vocabulary (OOV) issues by breaking rare words into subwords. To accommodate memory constraints, the vocabulary size was capped at $|V| = 20,000$, with special tokens automatically reserved for padding ($\langle \text{pad} \rangle$), start of sequence ($\langle \text{sos} \rangle$), end of sequence ($\langle \text{eos} \rangle$), and unknown words ($\langle \text{unk} \rangle$).

B. Encoder and Decoder Stacks

The implementation meticulously follows the canonical encoder-decoder structure (Figure 1). However, while the original architecture utilized $N = 6$ identical layers and a hidden dimensionality of $d_{\text{model}} = 512$, our checkpoint scaled this down to $N = 2$ layers and $d_{\text{model}} = 128$ to allow training on a 2 GB VRAM GPU.

1) *Encoder*: The encoder is composed of $N = 2$ identical layers. Each layer consists of two sub-layers: a multi-head self-attention mechanism, and a position-wise fully connected feed-forward network. We employ a residual connection [4] around each of the two sub-layers, followed by layer normalization [5]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$.

2) *Decoder*: The decoder is also composed of $N = 2$ identical layers. In addition to the two sub-layers found in the encoder, the decoder inserts a third sub-layer, which performs

multi-head cross-attention over the output of the encoder stack. To preserve the auto-regressive property, we modify the self-attention sub-layer in the decoder stack using an upper-triangular mask. This masking ensures that the predictions for position i can depend only on the known outputs at positions less than i .

C. Attention Mechanism

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors.

Scaled Dot-Product Attention

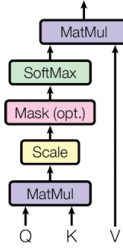


Fig. 2: The Scaled Dot-Product Attention mechanism, mapping queries, keys, and values to an output vector [1].

1) *Scaled Dot-Product Attention*: The core of the network relies on what is termed “Scaled Dot-Product Attention” (illustrated in Figure 2). We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values. In practice, this is computed simultaneously on a set of queries packed into a matrix Q , with keys and values packed into matrices K and V :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

For large values of d_k , the dot products grow exceedingly large in magnitude, pushing the softmax function into regions where it has extremely small gradients. To counteract this saturation effect, the dot products are explicitly scaled by $1/\sqrt{d_k}$.

2) *Multi-Head Attention*: Instead of performing a single attention function, it is beneficial to linearly project the queries, keys, and values h times with different, learned linear projections to d_k , d_k , and d_v dimensions, respectively.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (3)$$

where:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (4)$$

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. In our hardware-constrained implementation, we utilize $h = 4$ parallel attention layers (heads), where $d_k = d_v = d_{\text{model}}/h = 128/4 = 32$. As shown in Figure 3, this multi-head framework allows for highly parallelized processing.

D. Position-wise Feed-Forward Networks

Each layer in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (5)$$

In our implementation, the inner-layer dimensionality is set to $d_{ff} = 256$.

E. Positional Encoding

Because the model contains no recurrence and no convolution, we must inject information about the relative or absolute position of the tokens in the sequence using sine and cosine functions:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}}) \quad (6)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}}) \quad (7)$$

V. TRAINING OPTIMIZATION AND INFERENCE

A. Optimization Strategy

We utilized the Adam optimizer [6] with parameters $\beta_1 = 0.9$, $\beta_2 = 0.98$, and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training according to the Noam scheduling formula:

$$lr = d_{model}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5}) \quad (8)$$

This corresponds to increasing the learning rate linearly for the first 400 `warmup_steps`, and decreasing it thereafter proportionally to the inverse square root of the step number. We also applied dropout [7] for regularization.

B. Regularization via Label Smoothing

During training, we employed label smoothing [8] to prevent the model from becoming overconfident. The loss function $\mathcal{L}$ is defined as:

$$\mathcal{L} = - \sum_t \sum_v q_t(v) \log p_t(v) \quad (9)$$

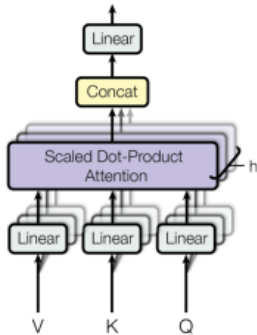


Fig. 3: Multi-Head Attention consists of several attention layers running in parallel [1].

where $q_t(v)$ is the smoothed target distribution at time step t , with $\epsilon = 0.1$:

$$q_t(v) = \begin{cases} 1 - \epsilon & \text{if } v = y_t \\ \frac{\epsilon}{|V|-1} & \text{otherwise} \end{cases} \quad (10)$$

C. Decoding Strategies

During inference, two decoding strategies were implemented:

- **Greedy Decoding:** An auto-regressive algorithm that selects the top-1 token with the highest probability at each step ($y_t = \arg \max_v p(y_t | y_{<t}, x)$).
- **Beam Search:** Keeps k best partial hypotheses at each step. To prevent a bias toward short sequences, we implement a length penalty with $\alpha = 0.6$. Our implementation defaults to a beam size of $k = 4$.

VI. RESULTS AND DISCUSSIONS

The model was trained for 20 epochs, and inference was tested using a 256-sample test benchmark. Computational throughput and empirical translation quality (BLEU-4) were measured on an NVIDIA RTX 2050 (2 GB VRAM).

TABLE I: Test Benchmark on 256 Samples (RTX 2050)

Decoding Method	BLEU-4	Runtime (s)	Samples/s
Greedy Search	7.22	47.44	5.40
Beam Search ($k = 4, \alpha = 0.6$)	7.22	103.43	2.48

A. Performance Analysis

The model achieved a BLEU-4 score of 7.22 on the test set, with a validation loss of 4.64. While a score of 7.22 is lower than production-grade models, it is an accurate reflection of the architectural constraints imposed on this experiment.

Transformers require massive parameter counts and vast amounts of data to converge to high-fidelity translations. The original Vaswani et al. model utilized over 65 million parameters and was trained on 8 P100 GPUs. In contrast, our constrained model ($d_{model} = 128, N = 2, h = 4$) drastically reduces the parameter count to fit within a 2 GB memory footprint. Consequently, the network lacks the representational capacity to map complex English-Hindi syntactic dependencies using only 30,000 synthetic pairs.

B. Computational Efficiency

Despite the limitations in absolute translation accuracy, the implementation proves highly efficient. Greedy decoding achieved a throughput of 5.40 samples per second, making it viable for interactive local inference. Beam search yielded the exact same BLEU score while reducing throughput to 2.48 samples per second, suggesting that for this specific configuration, the model's probability distributions are heavily peaked.

VII. CONCLUSIONS AND FUTURE WORK

This project successfully delivers a robust PyTorch implementation of the Transformer architecture applied to English-Hindi translation. By transcribing the original theoretical mechanisms into an executable pipeline—integrating SentencePiece BPE, label smoothing, masked multi-head attention, and Noam scheduling—it serves as a comprehensive educational replication of the architecture.

While the quantitative translation accuracy is fundamentally bottlenecked by hardware constraints and dataset size, the codebase establishes a functionally complete framework. Future iterations utilizing gradient checkpointing or distributed GPU training, combined with massive parallel corpora like the IIT Bombay English-Hindi Corpus, would elevate this foundational architecture to competitive production standards.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [2] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *arXiv preprint arXiv:1409.0473*, 2014.
- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” *arXiv preprint arXiv:1508.07909*, 2015.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [5] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2818–2826.